Benchmark

PDF edition of preserved repository document | docs/benchmark.md

Historical statements and test counts below retain their original scope. Consult the master report for current status.

Controlled benchmark infrastructure v1

Purpose and task

Measure accuracy, evidence-supported answers, trajectory validity, goal success, structural generalization and eventually data efficiency independently. There is no composite score. The first backend is signed decimal integer multiplication; the core state-transition implementation remains unchanged. Registry-backed dataset adapters supply sampling, identities, structure definitions, target validation and prediction assessment. Generic generation, persistence, splits, evaluation and result storage also work with non-arithmetic backends (covered by a test-only toy backend).

Schema and supervision

BenchmarkExample contains example_id, environment, problem, observation, initial_state, goal, target_answer, target_trajectory and metadata. Target trajectory stores transitions, without another full copy of observation/initial state. Metadata records seed, operand sizes, structure, split, generator version and measurable difficulty: carry_count, step_count, multiplication_steps, max_operand_digits and max_intermediate_value. carry_count counts digit operations producing nonzero carry. Optional difficulty_bounds filter those measurements, never subjective scores.

Model-facing views and export forms:

- answer_only: problem/observation; training export adds only answer.
- trajectory_supervised: also initial state/goal; training export adds trace/answer.
- experiential: observation/initial state/goal; no reference answer or trace, even in training export. Later learning must obtain feedback through environment interaction. Interactive learning is not implemented in this phase.

Evaluation always passes a deep-copied target-free view, regardless of regime. Models cannot access targets through the evaluator API. A deterministic reference executes the original environment from the observation rather than reading targets. Metadata/difficulty derived from target traces is also withheld from model views. The authoritative dataset contains targets and must never be handed directly to model training in the wrong regime. Future model integrations must enforce filesystem and procedural-knowledge access policies; Python APIs are not a security sandbox.

Generation, IID and OOD splits

TOML configs separately define counts and permitted structures for train, validation, test and optional ood_test. Explicit structures (such as 4x3) or per-split min_digits_a/max_digits_a/min_digits_b/max_digits_b ranges are supported. signed and include_zero are configurable. Each sampled target comes from the authoritative environment. Seeded split-specific RNGs and identity rejection generate numerical examples directly per split, rather than shuffling a giant dataset. Train guarantees at least one example per declared structure. IID examples use structures actually observed in train; OOD structures must be isolated, including swapped lengths.

Identity checks reject exact problem, operand-pair, example-id, and full-target-trace duplicates. Multiplication canonicalizes swapped operand pairs. Individual operand values may recur: 12×4 and 12×5 are distinct problems, not forbidden factor reuse. Shared procedural transition patterns are expected and are not trajectory leakage. The backend reconstructs every target to reject copied/tampered traces and metadata. Validation also checks split counts, membership, seed, version, difficulty and checksums. Failures are loud; records are never silently deleted. Training subsets are seeded, reproducible, nested prefixes and recorded as IDs in the manifest. Unavailable budgets are errors. Small subsets need not include every training structure.

Generation is rejection sampling, not balanced stratification. Finite small families can exhaust. In development_v1 all 45 unordered nonzero positive 1x1 pairs occur in train; IID test/validation therefore contain only 2x1 and 2x2. This distribution is reported, not a balanced per-family claim. Exhaustion raises after a bounded number

of attempts. Plan reserved per-structure quotas for final experiments if needed.

Metrics and denominators

Exact accuracy: type-strict correct final answers / all examples.

Trajectory validity: rule-consistent trajectories / supplied trajectories. Missing trajectories are excluded from this denominator; the supplied count is reported.

A valid incomplete prefix can have trajectory_valid=true but goal_success=false.

Goal success: complete verified final state AND correct claimed answer / examples.

Grounded answer rate: correct answer AND valid complete supporting trajectory bound to the same observation / examples. Correct answer-only predictions are ungrounded.

Unsupported answer rate: 1 - grounded rate. Empty denominators are null.

OOD accuracy is accuracy on ood_test; other split runs report it as null.

N*(alpha) uses only observed budget/accuracy points: minimum evaluated budget meeting alpha, or "not reached". DER = N*_baseline / N*_mindscape, or "not reached" if either threshold is unavailable. No interpolation, fabricated crossing, or deterministic reference data-efficiency claim. Alpha and observations are range-validated.

Results and reproducibility

JSONL splits and manifest.json contain exact config, generator/dataset versions, SHA-256 content hashes and nested subset IDs. Directory creation refuses overwrite.

The runner validates every split before any model call, including train contamination.

Model interface: predict(target_free_view) -> Prediction; the evaluator uses a backend, not model internals. Model exceptions and timeouts become failure records.

Each run creates a unique results/<timestamp_uuid>/ containing config.json, metrics.json, optional predictions.jsonl and summary.md. Config stores the full dataset manifest/hash, evaluation settings, Python/package versions and hardware. Metrics contain model/regime/budget/seed/split, separate rates, timings, call count and parameter count. Training time, checkpoint and hyperparameters are null for untrained references.

Prediction records include targets for post-evaluation auditing, correctness, trajectory, validity, goal status, groundedness and extensible error taxonomy.

Budget labels do not train or restrict this deterministic model's procedural knowledge. The runner is evaluation infrastructure; future training orchestration must train on the recorded subset and attach checkpoints/hyperparameters before scientific claims. Timestamps, IDs and timing vary across runs; data bytes, predictions and rates are reproducible. Results are retained locally and ignored by Git, not overwritten.

External historical references

This internal controlled benchmark is separate from historical general-model references. See historical_references.md. No external score is populated or compared. A multiplication accuracy cannot be compared with a score on another task.

Controlled numerical-claim revision

The revised study is documented in research_protocol.md and milestone_04.md. Prior singleton/oracle-assisted results are exploratory, not current evidence. Primary numerical-claim execution writes model proposals without correction; Regime C uses training-only scalar feedback and positive replay. All four conditions share capacity and the evaluator; remaining procedural/supervision/compute differences are explicit. All reported thresholds are censored; no data-efficiency advantage has been established. Use the research-specific CLI/config files for this study.